

Distributed Systems – Computer Assignment #1

Majid Sadeghinejad (810101459), Amin Tavanaie (810101396)
Spring 2026

Part 1: IPC Pipe Worker Interface

Architecture and Design

This system uses two independent Go programs to communicate. We have an **Interface process** and a **Worker process**. They talk to each other using special files called UNIX named pipes (FIFOs).

The **Interface process** is like the user's front door. It sits on your terminal, waiting for you to type commands. It checks if your input looks okay at a basic level. Then, it sends your request through a "request pipe" to the Worker. After sending, it waits patiently for a response to come back through a "response pipe." Once it gets the answer, it shows it to you on the screen. If the Worker process happens to stop or crash, the Interface is smart enough to try and reconnect to it.

The **Worker process** is the one that does the actual work. When it starts, it first checks if the "request" and "response" pipes exist. If they don't, it creates them so the Interface can find them. After that, it just sits there, constantly listening for new requests coming in through the request pipe. When a request arrives, it reads it, checks if the numbers and the operation are valid, and then does the math. Operations include ADD, SUB, MUL, DIV, and MIN. Once it's done, it sends the result back to the Interface through the response pipe. A key part of this design is that the Worker keeps running. It can handle many requests one after another without needing to be restarted, making it efficient. We use scanners to read input line by line, and loops to keep the processes running and to handle reconnections if needed. This separation means the Interface only worries about talking to the user and the pipes, while the Worker focuses only on doing the calculations.

Communication Protocol

The way these two programs talk is pretty straightforward. It's a request/response protocol. When the Interface wants the Worker to do something, it sends a **request**. A request looks like this: `OP A B`. Here, `OP` is the operation we want to perform (like `ADD`, `SUB`, `MUL`, `DIV`, or `MIN`), and `A` and `B` are the two integers we want to use for the calculation. For example, you might type `ADD 5 7` to add 5 and 7, or `SUB 10 3` to subtract 3 from 10. The Interface sends this whole thing as a single line, ending with a newline character, through the request pipe.

The Worker reads that line from the request pipe. It then breaks down the line to figure out the operation and the numbers. It checks if the operation is one it understands and if the numbers are valid. If everything is good, it does the math.

After the calculation, the Worker sends back a **response** as a string. If the calculation was successful, the response will look like `RESULT: <number>`, where `<number>` is the answer. For instance, if you sent `ADD 5 7`, the response might be `RESULT: 12`.

However, things can go wrong. If the Interface sends an invalid operation, like `SUM 5 7`, the Worker will send back an error message like `ERROR: Invalid operation`. Another common error is trying to divide by zero; in that case, the Worker would send `ERROR: Division by zero`. If the input is just malformed, like `ADD 5`, it would send `ERROR: Invalid arguments count`.

The programs are also designed to handle times when the communication might break. This could happen if the Worker crashes or the pipe gets messed up. In these situations, the Interface will notice that it can't talk to the Worker anymore and will try to reconnect, giving the Worker a chance to start up again and resume its job.

Part 2: Context Switching and Concurrency

In this part, I wrote a test program to see how goroutines act in different situations and how context switching impacts the overall performance and system efficiency.

App Structure

I structured the code to run a comprehensive benchmark by setting up exactly 1024 tasks to process. I chose 1024 because it divides perfectly among 1, 2, 4, 8, 16, 32, and 64 goroutines. I used nested loops to sequentially test every combination of CPU cores (GOMAXPROCS), number of goroutines, and workload types.

Inside the loops, I divide the 1024 tasks equally among the active goroutines for that specific test. To measure performance accurately, I used the Go `time` library to record exactly when the jobs start and finish. Since the goroutines execute concurrently, I used a `sync.WaitGroup`. Each time a goroutine starts, I add it to the wait group, and when it finishes, it signals the group. The main program simply waits at the end until everyone is done to calculate the total execution time. Meanwhile, each goroutine tracks the exact latency of its own specific tasks so I can calculate the minimum, maximum, and average latency at the end.

There are two main workloads I wrote for this test:

- **CPU-Bound:** This is a pure computational task. It is essentially a tight loop calculating math square roots 100,000 times without ever stopping. It simulates a heavy task that needs 100 percent of the processor.
- **Mixed-Contention:** This task mimics a real-world web server. It does some minor math, but every 500 steps it tries to increment a shared counter variable. Because multiple goroutines might try to change this variable simultaneously, I wrapped it in a `sync.Mutex` lock. This means if one goroutine is updating the counter, all others must wait in line. Finally, this task includes a temporary `time.Sleep` operation to simulate I/O operations (like waiting for a network request), which forces the goroutine to pause completely.

Test Results

Goroutines	Workload	TotalTime(ms)	Throughput/s	AvgLat	MinLat	MaxLat
1	CPU-Bound	36	28235.46	0.0350	0.0310	0.1230
1	Mixed	594	1724.21	0.5790	0.5120	0.7260
2	CPU-Bound	37	27590.39	0.0540	0.0310	18.6120
2	Mixed	298	3441.33	0.5800	0.5060	0.9020
4	CPU-Bound	34	30473.28	0.0320	0.0300	0.1020
4	Mixed	147	6942.41	0.5760	0.5060	0.6370
8	CPU-Bound	37	27504.55	0.0360	0.0310	0.1160
8	Mixed	73	13943.53	0.5730	0.5050	0.7690
16	CPU-Bound	35	29156.07	0.0340	0.0300	0.1020
16	Mixed	37	27852.51	0.5730	0.5040	0.6340
32	CPU-Bound	33	30692.34	0.0320	0.0300	0.0970
32	Mixed	18	57128.65	0.5550	0.5040	0.7350
64	CPU-Bound	33	31280.99	0.0310	0.0300	0.1200
64	Mixed	9	112928.72	0.5460	0.5040	0.8470

Table 1-Table for GOMAXPROCS 1

Goroutines	Workload	TotalTime(ms)	Throughput/s	AvgLat	MinLat	MaxLat
1	CPU-Bound	32	31567.91	0.0310	0.0300	0.0930
1	Mixed	594	1722.60	0.5800	0.5150	0.6830
2	CPU-Bound	22	45895.61	0.0430	0.0370	0.0920
2	Mixed	298	3437.57	0.5810	0.5150	0.6830
4	CPU-Bound	23	43983.58	0.0450	0.0400	0.1080
4	Mixed	165	6201.91	0.6410	0.5090	2.9360
8	CPU-Bound	17	60004.25	0.0330	0.0310	0.1070
8	Mixed	73	14000.64	0.5710	0.5100	0.6590
16	CPU-Bound	17	59611.13	0.0330	0.0310	0.1090
16	Mixed	37	28048.55	0.5690	0.5040	0.6260
32	CPU-Bound	17	59444.31	0.0330	0.0310	0.1570
32	Mixed	18	56041.52	0.5670	0.5040	0.6270
64	CPU-Bound	16	62412.70	0.0310	0.0300	0.0710
64	Mixed	9	113354.79	0.5530	0.5040	0.6790

Table 2-Table for GOMAXPROCS 2

Goroutines	Workload	TotalTime(ms)	Throughput/s	AvgLat	MinLat	MaxLat
1	CPU-Bound	32	31584.55	0.0310	0.0300	0.0790
1	Mixed	592	1729.43	0.5780	0.5110	0.6650
2	CPU-Bound	17	61047.01	0.0320	0.0300	0.0900
2	Mixed	295	3470.73	0.5760	0.5250	0.6630
4	CPU-Bound	9	110400.34	0.0350	0.0340	0.1450
4	Mixed	148	6924.91	0.5770	0.5190	0.6470
8	CPU-Bound	8	132348.90	0.0390	0.0300	1.0320
8	Mixed	74	13881.59	0.5760	0.5100	0.6550
16	CPU-Bound	6	161603.41	0.0400	0.0310	3.3020
16	Mixed	37	27712.38	0.5760	0.5080	0.6860
32	CPU-Bound	6	161100.21	0.0420	0.0310	3.5130
32	Mixed	19	55217.41	0.5770	0.5050	0.7140
64	CPU-Bound	6	172941.34	0.0400	0.0310	1.7430
64	Mixed	9	108710.65	0.5750	0.5050	0.7770

Table 3-Table for GOMAXPROCS 8 (Max CPU)

Analysis

1. What happens to execution time with more goroutines?

With a pure CPU-bound task, increasing the number of goroutines only decreases the execution time if there are available physical CPU cores (when GOMAXPROCS is higher than 1). For example, when my GOMAXPROCS was 1, the total time stayed rigidly around 33-37ms whether I used 1 goroutine or 64. However, when I set it to 8 cores, jumping from 1 to 16 goroutines slashed the total time drastically from 32ms down to just 6ms. This happens because the work can be truly divided across separate cores. With the mixed workload, increasing goroutines reduces the execution time almost magically across the board, even on just 1 CPU core. When 1 goroutine goes to sleep, the Go scheduler immediately hands the CPU over to another active goroutine rather than letting the machine idle. By 64 goroutines, the execution time dropped from 594ms down to roughly 9ms in every test simply by hiding the sleep latency.

2. Does increasing goroutines always increase throughput?

No, definitely not. There is a breaking point. If you look at the Mixed-Contention test at 64 goroutines using 8 CPU cores, the throughput is actually slightly lower (108,710/s) than when doing the exact same test on just 1 or 2 CPU cores (112,928/s and 113,354/s).

This goes against the normal assumption that more hardware equals more speed. It performed worse because all 8 independent hardware threads were actively colliding into the exact same Mutex lock at the exact same physical moment. This forces the processor to constantly invalidate hardware cache lines and makes the threads fight to acquire the lock. The machine spends more time managing the conflict around the lock than actually doing the work.

3. The difference between GOMAXPROCS=1 and GOMAXPROCS=NumCPU (8)

When GOMAXPROCS is set to 1, the Go runtime only binds to a single physical OS thread. The execution here is strictly concurrent, not parallel. The Go scheduler rapidly slices time and swaps goroutines perfectly, which is excellent for I/O-bound work (like our mixed workload) but useless for heavy math.

When it equates to 8, the Go runtime leverages 8 separate physical OS threads interacting with the M1 chip simultaneously. This is true parallelism. The difference is

monumental for pure calculation. Our math workload went from a throughput of 31,000 tasks/second on 1 core to 172,000 tasks/second on 8 cores.

4. In which workload is scheduling and switching effects more visible?

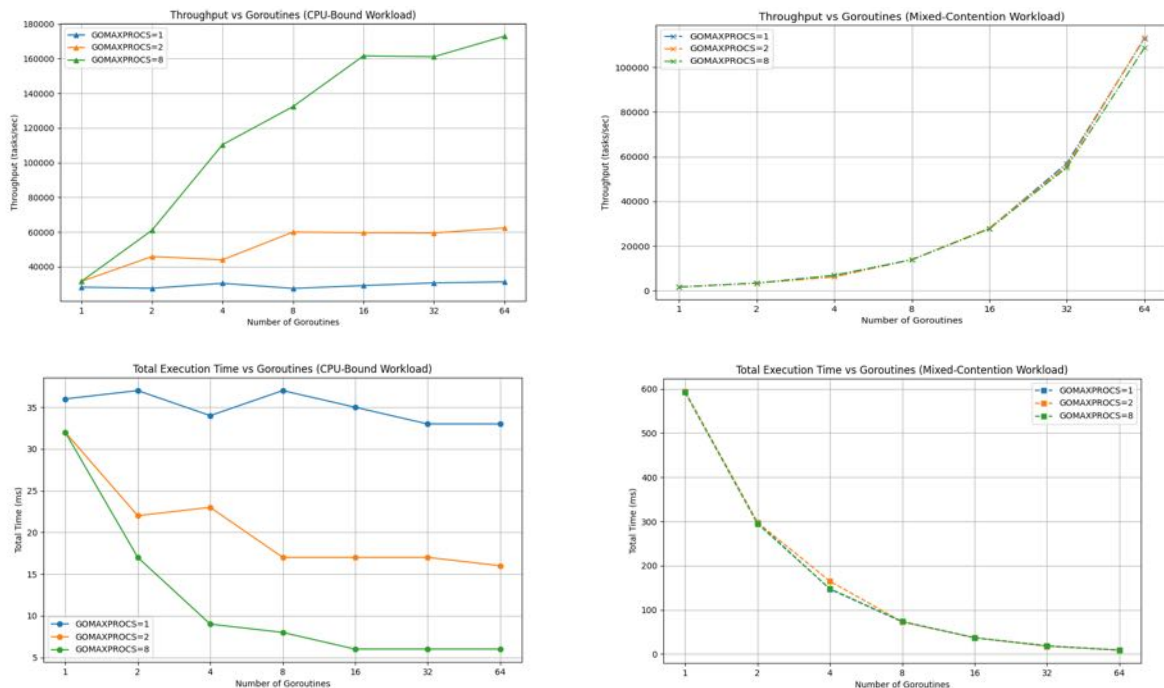
They are massively visible in the Mixed-Contention workload. Due to the sleep timers and Mutex locks, heavy context switching occurs. When a goroutine hits the sleep command, it detaches from the thread. The Go scheduler completely bypasses the heavy operating system limitations and instantly places another ready goroutine onto the thread in user-space. This makes lightweight user-space switching extremely cheap and explains why having 8 cores vs 1 core didn't heavily impact the mixed-workload time at 64 goroutines.

5. When is it useless to increase concurrency?

Concurrency becomes actively harmful in two scenarios shown in the data. First, in pure CPU workloads, adding more goroutines than you have physical cores stops providing benefits. As seen in the 8-Core CPU test, adding more goroutines beyond 16 did not lower the 6ms total time, but it drastically ruined the Maximum Latency metric (spiking it up to 3.5ms). This indicates that the scheduler just starts wasting cycles swapping active threads in and out.

Second, it is useless to increase concurrency when there is extreme contention over a shared resource. In our mixed test, pushing past 32 goroutines on 8 cores throttled the system downward due to lock contention on the Mutex. At that point, adding more workers just makes the line to acquire the lock longer.

Graph Analysis



By looking at the generated graphs, we can visually confirm the points made above:

- **Total Execution Time (CPU-Bound):** It is very clear that the 1-core line is completely flat and doesn't change with more goroutines. However, the 8-core line drops sharply and then flattens out.
- **Total Execution Time (Mixed-Contention):** The slope of this graph drops aggressively for all core configurations, showing just how well Go's concurrency masks sleep delays and wait times.
- **Throughput (CPU-Bound):** This acts as the exact inverse of the execution time graph, showing incredible scaling and speed on the 8-core hardware tests.
- **Throughput (Mixed-Contention):** This graph clearly shows the contention issue we discussed. You can visibly see the 8-core line reach its peak and then start to dip downwards as the number of goroutines increases due to the heavy traffic fighting over the Mutex lock.

Go Calculator Service Documentation

Architecture Overview

Handler-based architecture built with the `net/http` package.

The service exposes the following endpoints:

- **/health**: This endpoint is dedicated to health checks, allowing users to verify the operational status of the service.
- **/compute**: This endpoint handles basic arithmetic computations. It supports the following operations: add, mul, sub, div

Operations are specified using URL query parameters. The Docker image for this service is built on `golang:1.21-alpine` to ensure a minimal footprint.

Build Steps

The service is packaged as a Docker image. The build process involves the following steps:

1. Copy project files into a Docker container.
2. Build the Go binary using the command: `go build -o server main.go`.
3. Expose port 8080 to allow external access to the service.
4. Set the generated binary as the container's entrypoint to run the application upon container start

Run Instructions

To build and run the Go Calculator Service, follow these steps:

1. **Build the Docker image**: `bash docker build -t go-calculator .`
2. **Run the Docker container**: `bash docker run -p 8080:8080 go-calculator`

The service will be exposed on port 8080.

Host-to-VM Connection: Access the service from your host machine via `http://<VM-IP>:8080`, where `<VM-IP>` is the IP address of your virtual machine.

Sample curl Commands

Below are examples of `curl` commands to interact with the Go Calculator Service:

- **Health Check**: `bash curl http://<VM-IP>:8080/health`

- **Example Computation (Addition):**

```
bash curl "http://<VM-IP>:8080/compute?op=add&a=5&b=3"
```

This command will compute $5 + 3$ and return the result. You can replace op, a, and b with other operations and operands as need.

Execution proof

```
root@tavana:~# docker build -t go-compute-service .
[+] Building 17.4s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/golang:1.21-alpine
=> [internal] load .dockerignore
=> [internal] transfer context: 0B
=> [1/9] FROM docker.io/library/golang:1.21-alpine@sha256:2414835886a3c42089654c0b26ae75b1d158088d65f083c7f995527fadc94
=> resolve docker.io/library/golang:1.21-alpine@sha256:2414835886a3c42089654c0b26ae75b1d158088d65f083c7f995527fadc94
=> [internal] load build context
=> transferring context: 1.01kB
=> CACHED [2/4] WORKDIR /app
=> [3/4] CMD
=> [4/4] RUN go build -o server main.go
=> exporting to image
=> exporting layers
=> exporting manifest sha256:c18f80bc4542a7be0219720c5a0840f20c5f0f01104f3a0fa7d439c1f
=> exporting config sha256:1dec75a787adceaf70ff10b0b0b4dc3c3a0f046359c3010070bc2400800
=> exporting attestation manifest sha256:13a0093c7a7b177c7f319dc507b04500667a0b3e11eb0e121350ba4308500a
=> exporting manifest list sha256:0a48103eeaaa49682770d1b1a7ac7fca0ffec4ac049ec1391004f52d1a730e0
=> naming to docker.io/library/go-compute-service:latest
=> pushing to docker.io/library/go-compute-service:latest
root@tavana:~# docker run -d -p 8080:8080 go-compute-service
f190439cec11dd14ae363dd42212368b369754c6816d4e6f39a2ed791d91c57
root@tavana:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
f190439cec11   go-compute-service   "/server"                36 seconds ago Up 36 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   brave_sinoussi
```

```
» curl "http://192.168.56.10:8080/compute?op=mul&a=5&b=7"
curl "http://192.168.56.10:8080/compute?op=add&a=5&b=7"
curl "http://192.168.56.10:8080/compute?op=pow&a=5&b=7"
curl "http://192.168.56.10:8080/compute?op=add&a=5&b=e"
curl "http://192.168.56.10:8080/compute?op=div&a=5&b=0"

{"operation": "mul", "a": 5, "b": 7, "result": 35}
{"operation": "add", "a": 5, "b": 7, "result": 12}
invalid operation
b must be number
division by zero
» curl "http://192.168.56.10:8080/health"
service is healthy%
```